Teoría S2 (60 min) · resumen y glosario

Corresponde a las diapositivas 4–17. Proyecto de trabajo: `04-Proyecto-Base/` y su documento `docs/api-y-contenedor.md`.

1. Del modelo al servicio (diap. 4)

Un modelo guardado en disco no es un servicio. Para que otra persona o programa lo use hace falta: un **contrato** de entrada, **validación**, **errores** que se entiendan, **registros**, una **imagen** que se construya igual en cualquier sitio y una **comprobación automática** de todo eso. El modelo es el mismo de S1; cambia lo que lo rodea.

2. El contrato (diap. 5–6)

Son las 13 variables de FS de un cliente en una fecha de corte, cada una con tipo y límites. Los límites salen de **cómo se construye FS**, no de lo que se vio al entrenar: un cliente con más facturas que cualquiera de train es un cliente legítimo. Un campo que no es del contrato (el objetivo `y`, `cliente_id`, una fecha) se **rechaza**, no se ignora.

El contrato no se escribe dos veces. El modelo se guarda en formato MLflow y lleva su **firma** (nombre, tipo y obligatoriedad de cada entrada). Al arrancar, la API compara tres cosas: la firma, `metadatos.json` y el esquema de `Cliente`. Si no coinciden, el proceso sigue vivo pero **no está listo** (503) y dice qué variable falta o sobra. Las pruebas vigilan lo mismo antes de desplegar.

3. Anatomía de una petición (diap. 7)

Seis estaciones: llega la petición → el *middleware* toma o crea el `X-Request-ID` → se **valida** contra el esquema (422) → el **servicio** ordena las columnas, llama al modelo y calcula las advertencias (503 si no hay modelo) → se **responde** → se escribe **una línea de registro**. Cada etapa puede decir «no» con un código distinto, y un fallo no controlado acaba en el *middleware* como 500. El modelo solo se llama si la validación pasó; por eso, con el esquema roto, una petición con un campo desconocido recibe 422 aunque el servicio no esté listo.

4. Rechazar o avisar (diap. 8–9)

- **Se rechaza (422)** lo *imposible* o *incoherente*: recencia negativa, más facturas en 90 días que en total, `ticket_medio` que no es `monto_total / frecuencia`. Casi seguro es un error de quien llama.
- **Se avisa (200 con `advertencias`)** lo *posible pero sin respaldo*: un ticket medio de 5 000 libras cuando el modelo solo vio hasta 1 295; un cliente con una sola factura (no hay intervalo medio, el modelo usa la mediana de train).
- Rechazar lo segundo dejaría sin predicción a un cliente real; callarlo escondería que el modelo extrapola.

5. Cuatro códigos (diap. 10)

Código	Qué significa	Quién actúa
200	Predicción (con o sin advertencias)	—
422	La petición no cumple el contrato	Quien llama
503	El servicio no está listo (sin modelo, o contrato roto)	Quien opera el servicio
500	Fallo interno inesperado; no cuenta nada interno, el traceback va al registro	El desarrollador, con el `id_peticion`

6. Vivo no es lo mismo que listo (diap. 11)

`/salud` pregunta si el **proceso** responde: lo usa un orquestador para decidir si lo **reinicia**. `/listo` pregunta si el **modelo** está cargado y coincide con su contrato: lo usa para decidir si le **envía tráfico**. Un servicio sin modelo está vivo y no está listo; reiniciarlo no lo arregla.

7. Registros (diap. 12)

Una línea JSON por petición, por la salida estándar (`severity`, `message`, `id`, `ruta`, `codigo`, `ms`): así la recoge Docker, Actions o Cloud Logging sin configuración. El `id` une lo que vio quien llamó con lo que dice el registro. **Nunca se registran los datos del cliente.** Una lección real de la solución de referencia: una primera versión usaba `logging` sin manejador; con `TestClient` se veían líneas y con `uvicorn` de verdad no salía ninguna. Por eso hay una prueba que arranca `uvicorn` como lo hace el contenedor.

8. Cuatro caminos, una probabilidad (diap. 13)

El mismo modelo se puede ejecutar por el paquete, el `joblib`, el formato MLflow y la API (local o en un contenedor). Si es el mismo, las cuatro probabilidades son idénticas hasta el último decimal. Si no, algo cambia entre entrenar y servir: otra versión de una librería, otro orden de columnas, otro preproceso. `python -m ia_mlops.modeling.comparar` lo mide con filas reales de test.

9. Imagen y contenedor (diap. 14)

La **imagen** es una receta congelada, de solo lectura; el **contenedor** es esa imagen en ejecución, aislada. Se construye una vez y se ejecuta donde se quiera: portátil, Codespace, CI, Cloud Run en S3.

10. El Dockerfile (diap. 15–16)

- `python:3.11-slim`: el mismo Python que el entorno del curso.
- Primero las dependencias (`requirements-api.lock`, versiones exactas) y después el código: la capa de dependencias se reutiliza si solo cambia una línea de código.
- Solo entra lo que sirve el modelo: `ia_mlops/`, `models/modelo_mlflow/`, `models/metadatos.json`. Sin datos, SQL, cuadernos ni pruebas.
- Usuario `10001` (no root); `uvicorn` en `0.0.0.0:8080` (en `127.0.0.1` solo se alcanzaría desde dentro del contenedor); `HEALTHCHECK` sobre `/salud`.

- `.dockerignore` deja fuera todo lo demás. Un patrón como `*.pyc` solo vale en la raíz: para cualquier carpeta, `**/*.pyc`.
- La imagen construida ocupa 826 MB en disco (pandas, scipy y scikit-learn pesan), de los cuales unos 183 MB son «contenido» según `docker image ls`.

11. El primer workflow (diap. 17)

`deploy/github-actions/ci.yml` es una plantilla: se copia a `.github/workflows/ci.yml`. En cada `push` o `pull_request` a `develop` o `main`: instala dependencias, ejecuta el pipeline, pasa las pruebas, construye la imagen, arranca el contenedor y comprueba `/listo`, `/modelo`, una predicción, que un campo fuera del contrato da 422 y que el proceso corre como `10001`. Permisos mínimos (solo lectura), sin secretos y sin publicar nada: publicar y desplegar llegan en S4.

Qué está comprobado

Las pruebas, la imagen y el workflow se verificaron en Linux y en GitHub Actions. **Sin probar:** el Codespace del curso (`.devcontainer`), Docker en un equipo propio y los comandos en Windows. Ver la sección «Verificación» de `docs/api-y-contenedor.md`.

Glosario

- **API:** programa que recibe peticiones HTTP y devuelve respuestas.
- **Contrato:** nombre, tipo y límites de lo que la API acepta y devuelve.
- **Firma del modelo:** descripción de entradas y salida guardada junto con el modelo.
- **Esquema (Pydantic):** la clase que valida el cuerpo de la petición.
- **422 / 503 / 500:** petición inválida / servicio no listo / fallo interno.
- **`/salud` y `/listo`:** el proceso responde / el servicio puede atender tráfico.
- **Registro (log):** línea de texto que el servicio escribe por cada evento.
- **`X-Request-ID`:** identificador que une la respuesta con el registro.
- **Imagen / contenedor:** receta congelada / receta en ejecución.
- **Capa (de imagen):** paso del Dockerfile cuyo resultado se guarda y se reutiliza si no cambió.
- **Usuario no root:** el proceso no tiene permisos de administrador dentro del contenedor.
- **CI:** comprobación automática en cada cambio (aquí, GitHub Actions).
- **`cloudpickle`:** formato de serialización del modelo; puede ejecutar código al cargarse, por eso solo se cargan modelos propios.